# Historial de versiones

Qué cambia en cada versión del ambiente, del agente y del formato de las ejecuciones. Cada ejecución de entrenamiento guarda en su `manifest.json` la versión del ambiente y el commit de git con el que se hizo, así siempre se sabe con qué reglas se entrenó un modelo.

Regla del equipo: un cambio en las reglas o en las constantes del ambiente sube su versión (0.1 → 0.2) y se anota aquí. Los modelos entrenados con versiones distintas del ambiente no se comparan entre sí.

## Entregas del proyecto (etiquetas de git)

### v1.1.1 · 2026-11-13

- Entrega final: nombres completos del equipo en el informe y en el índice de entregables; la web se presenta como **Alpha Centauri** (antes "PROJECT ELYNDRA"; Elyndra sigue siendo el nombre del planeta ficticio de la escena). PDF regenerados con enlaces a `v1.1.1`.
- El viento en vivo pasa a ser un **modo del simulador** (selector "Replay grabado / Viento en vivo" bajo las fichas de modelo; `#/vivo` redirige allí) y desaparece del menú. La API acepta `DELETE` en CORS para que el navegador cierre la sesión.

### v1.1.0 · 2026-11-11

- **Viento en vivo** ( `#/vivo` ): la API simula paso a paso con el modelo congelado ( `POST /sesiones` , `POST /sesiones/{id}/paso` , `DELETE /sesiones/{id}` ; sesiones en memoria, 20 como máximo, 15 minutos de vida) y la consola de misión deja cambiar la intensidad y la dirección del viento con una barra mientras el cohete vuela; el cambio entra en el siguiente paso del agente. No es un entrenamiento nuevo: es un modo de servicio sobre los modelos ya versionados.
- **Pantalla completa** de la escena 3D en el simulador y en la misión en vivo (botón, tecla F, barra flotante con reproducir, pasos y, en vivo, los deslizadores del viento; Esc sale). Donde no existe la API Fullscreen se fija la escena a la ventana.
- Paquete de entregables: entregables.md (rúbrica → evidencia) y `python entregables.py` , que genera los PDF de `entregables/` a partir de los Markdown del repositorio. 113 pruebas. Las reglas de los ambientes no cambian (0.1.1, 0.4, 0.5).

### v1.0.0 · 2026-11-10

- Cierre del proyecto: tres niveles (N1 0.1.1, N2 v1 0.4, N3 v1 0.5), 23 ejecuciones en 6 experimentos, pipeline Prefect de 8 etapas con 21 evidencias, API y web públicas, catálogo v15, monitoreo, incidente, intervención, rollback demostrado, informe final, guion de la defensa y test reservado.

### v0.2.0 · 2026-11-03

- Hito 1: ambiente 0.1 con ficha, experimentos 1 (Q-learning) y 2 (PPO), MLflow, API, web y despliegue en contenedor.

## Ambiente RocketLanding3D-N3-v0 (nivel 3)

### 0.3 · 2026-11-05

- Nivel 3 = nivel 2 con condiciones de operación exigentes: viento a 10 m sorteado entre 0 y 15 m/s (antes 0–10) y ráfagas de 3 m/s (antes 1,5). Ficha: ambiente-v0.3.md. Clase `RocketLandingN3Env` en `rocket_env/nivel3.py`, hereda del nivel 2.
- Misma observación (25 valores) y acción que el nivel 2: misma familia de modelos. Un modelo de N2 juega en N3 sin adaptador (`hacer_entorno`, `POST /evaluar` con `ambiente`), lo que mide la degradación (`python incidente.py`).
- `VIENTO_MAX` y `RAFAGAS` pasan a ser constantes de clase de `RocketLanding3DEnv` (10 y 1,5): los niveles 1 y 2 no cambian (mismas semillas, mismos sorteos, mismas evaluaciones guardadas).

## Ambientes RocketLanding3D-N2-v1 y RocketLanding3D-N3-v1 (niveles 2 y 3 con pasarela)

### 0.4 (N2-v1) y 0.5 (N3-v1) · 2026-11-06

- El equipo cambió el obstáculo de los niveles 2 y 3: en vez de tres torres, una **pasarela** horizontal de 90 m de radio a 180 m de altura que cubre la plataforma y todo el cuadrado de salida, con una sola abertura circular de 16 m de radio desplazada 30 m del centro (en (28, 12) m), sostenida por cuatro pilares en el borde. Chocar con la pasarela o con un pilar es "colisión" (−100). Ficha: ambiente-v0.4.md; clases en `rocket_env/pasarela.py`.
- Decisión nueva en dos tiempos: alinearse con la abertura antes de llegar a la pasarela y, debajo, trasladarse hasta la plataforma mientras se frena. Observación de 16 + 4 valores (posición relativa del objetivo de la fase: la abertura por encima de la pasarela y la plataforma por debajo; altura hasta la pasarela; margen hasta el borde de la abertura): familia nueva. `TRANSFERENCIA_ENTRADAS` indica cómo hereda un modelo del nivel 1 los pesos de posición para esas entradas. El potencial de la recompensa apunta a la abertura mientras el cohete está por encima de la pasarela y castiga acercarse a ella fuera de la abertura (hasta 8 puntos en los 20 m previos).
- 0.5 = 0.4 con viento de 0 a 15 m/s y ráfagas de 3 m/s (como pasaba de 0.2 a 0.3).
- Los ambientes con torres (0.2 y 0.3, ids `-v0`) siguen registrados: los modelos entrenados en ellos son históricos y se pueden evaluar igual. `rocket_env.VIGENTES` dice el ambiente actual de cada nivel; la API lo marca con `vigente`.
- Dificultad medida: el recomendado del nivel 1 a ciegas (`AdaptadorN1`) baja de 99 % a 5 % (190 colisiones de 200).

## Ambiente RocketLanding3D-N2-v0 (nivel 2, retirado el 2026-11-06)

### 0.2 · 2026-11-05

- Nivel 2 = nivel 1 + tres torres alrededor de la plataforma (a 28 m del centro, 120° entre sí, radio 7 m, alturas 420/360/300 m). Chocar con una torre es el resultado nuevo "colisión" (−100). Ficha: ambiente-v0.2.md. Clase `RocketLandingN2Env` en `rocket_env/nivel2.py`, hereda la física.
- Observación de 16 + 3 × 3 = 25 valores (por torre: posición relativa x, y y altura que queda por encima); acción igual. Es una familia nueva de modelos: los del nivel 1 no juegan aquí salvo con `AdaptadorN1` (recorta la observación a los 16 valores del nivel 1), que sirve para medir la dificultad del nivel.
- Potencial de la recompensa = el del nivel 1 − 8 × (1 − d/15 m) por cada torre a menos de 15 m de la superficie y por debajo de su cima + 20 m. Lo demás (pasos, aterrizaje, viento, combustible) no cambia.
- Cada ejecución guarda en su manifest `ambiente.id`, `nivel` y `obstaculos`; la API y la CLI crean el ambiente que dice el manifest. Los manifests anteriores al nivel 2 no traen `ambiente.id`: se interpretan como `RocketLanding3D-v0`.

## Ambiente RocketLanding3D-v0

### 0.1 · 2026-10-28

- Primera versión documentada en ambiente-v0.1.md.
- Pasos de 20 m de desplazamiento vertical (o 2 s), máximo 50 pasos por época, salida a 600 m.
- Zona de vuelo: cilindro de 300 m de radio y 1200 m de techo. Plataforma de 10 m de radio.
- Aterrizaje: |vz| ≤ 3 m/s, v_h ≤ 2 m/s, inclinación ≤ 10°, distancia ≤ 10 m. Recompensas finales +100 / −50 / −100.
- Observación de 16 valores (incluye inclinación); acción continua de 5 valores.
- Atmósfera ISA con suelo aleatorio por época; viento con ley 1/7 y ráfagas; perturbación `options={"viento": v}`.

### 0.1.1 · 2026-11-03

- "sin pasos" (agotar los 50 pasos sin tocar el suelo) se informa como `truncated=True` en vez de `terminated`, como pide la guía para el límite externo de la época. La recompensa (−100) y todas las reglas son las mismas.
- Tercer número de versión = señales que no cambian las reglas. `RocketLanding3DEnv.compatible(v)` compara solo mayor.menor: los modelos entrenados con 0.1 siguen siendo válidos y comparables en 0.1.x.

## Agente Q-learning

### 0.1 · 2026-10-28

- Dos tablas Q: motor (altitud × velocidad vertical × 6 acciones de empuje relativo al peso) y dirección (deriva prevista en x × en y × 5 direcciones con inclinación lateral 0,10).
- ε de 1 a 0,02 en el 60 % de las épocas; α de 0,1 a 0,01; γ 0,99; evaluación cada 5 % de las épocas y se conserva la mejor versión.

## Agente PPO (experimento 2)

### 0.1 · 2026-10-28

- Red MLP 128×128 (Stable-Baselines3, PyTorch) sobre la acción relativa v2: 3 valores en [−1, 1] (empuje = 1 + $a_0$, máximo ×1,6; inclinación = $a_1$, $a_2$ × 0,12). 600 000 pasos en 8 entornos, evaluación sin ruido cada 20 000 pasos, se conserva la mejor versión.
- Historia: la acción cruda (`--accion cruda`) y la relativa v1 (inclinación ±0,3) no aterrizan; ver docs/experimento-2.md.

## Formato de las ejecuciones (`resultados/runs/<id>/`)

### 1 · 2026-10-28

- `manifest.json`, `modelo.npz`, `entrenamiento.json`, `vuelos.json.gz`; índice en `resultados/registro.json`.
- Mismo día, ampliación compatible: `experimento`, `agente.clase/nombre` y `archivos.modelo` como lista en el manifest; `episodios.csv` (dataset) y `evaluacion.json` (evaluación detallada) en cada ejecución; registro en MLflow (`mlflow.db` + `mlruns/`).
- 2026-11-05, ampliación compatible: el catálogo lleva un recomendado y un anterior por nivel (`niveles.N1`, `niveles.N2`); `recomendado` / `anterior` de primer nivel siguen siendo los del nivel 1. `registro.json` y el manifest guardan `nivel`; `ambiente.id` y `ambiente.obstaculos` en el manifest. `OUTCOMES` global = los del nivel 2 (los cinco del nivel 1 conservan su índice en `entrenamiento.json`).
- 2026-11-03, ampliación compatible: `configs/<run_id>.json` (hipótesis, cambio frente al baseline, presupuesto, estado seleccionado/rechazado/referencia y decisión; también como etiquetas `diseno.*` y `presupuesto.*` de la run de MLflow), `manifests/<run_id>.json` con el formato de la sección 11 de la guía (`python manifiestos.py`: commit completo, SHA-256 del modelo, `model_uri`, suite, nivel) y suites de evaluación con nombre en `configs/suites/` (`suite-v1`; la reservada no lleva semilla en el repo). Los manifests nuevos guardan el SHA completo del commit.
- 2026-10-30, ampliación compatible: `sha256` en el manifest (huella de la configuración resuelta y de cada archivo de `archivos`), comprobable con `python cli.py verificar`. Las cuatro ejecuciones anteriores recibieron sus huellas ese día, con una nota que lo indica; sus copias en `mlruns/` conservan el manifest sin huellas.